Assignment 2: Generative Models and Sequence-to-Sequence Learning

Blokland, A. Ghaneh, T. Rickert , P.

December 8, 2025

Introduction

Deep learning builds models that learn patterns from data through many layers of computation. Generative models such as VAEs and GANs learn a latent space and use it to create new images. VAEs sample from a shaped latent space; GANs use a generator–discriminator game to sharpen the outputs. For sequences, RNNs encode an input sequence and decode it into another one, which makes them useful for tasks such as arithmetic in text or images. In this assignment, we addressed two objectives. First, we studied examples of VAE and GAN architectures, trained them on our own dataset (containing anime faces), studied their behaviors, and optimized them to sample new, unseen images from the respective latent spaces. Second, we built and compared several sequence-to-sequence RNN models that handle addition/subtraction across different input–output forms: text-to-text, image-to-text, and text-to-image.

1 Task 1: Generative Models

The first task included studying the models. We went beyond studying the code and investigated the underlying mathematical principles. As this is somewhat beyond the scope of the assignment, we have included it in Appendix A.

1.1 Dataset Selection and Preprocessing

For our dataset, we decided to use a dataset that contains anime faces. The dataset contains 135,027 JPG images with a size of 96×96 pixels. We shuffled it, sampled 22,000 images, and converted them to a 4D zipped numpy array of shape (22000, 96, 96, 3). The pixel values were normalized before being fed to the algorithms. The discussion in Appendix B explains why we prefer 96×96 over 64×64.

1.2 Convolutional Auto-Encoder (CAE)

We began with the CAE to observe reconstruction-loss behavior and establish a baseline for tuning hyperparameters before moving to the VAE. Our search strategy used an expanded grid over three hyperparameters (latent dimension L, number of upsampling layers U, and number of filters/channels C); after identifying the best combination, we tested it across different batch sizes. The detailed reasoning and time-accuracy trade-offs are mentioned in Appendix B. After 100 epochs, the reconstructed images looked very similar to the original ones.

1.3 Variational Auto-Encoder (VAE)

For the VAE, the number of tuneable hyperparameters was too large for a grid search to optimize. Therefore, we used a trial-and-error process leading to the set of hyperparameter values in Table 1.

Table 1: Hyperparameters for the VAE model

Parameter	Epochs	Batch size	Latent dim	Filters	Kernel	Upsample	Coef	β	LR	β_1
Value	350	32	1024	256	3	5	0.9	1	1×10^{-4}	0.9

1.4 Generative Adversarial Network (GAN)

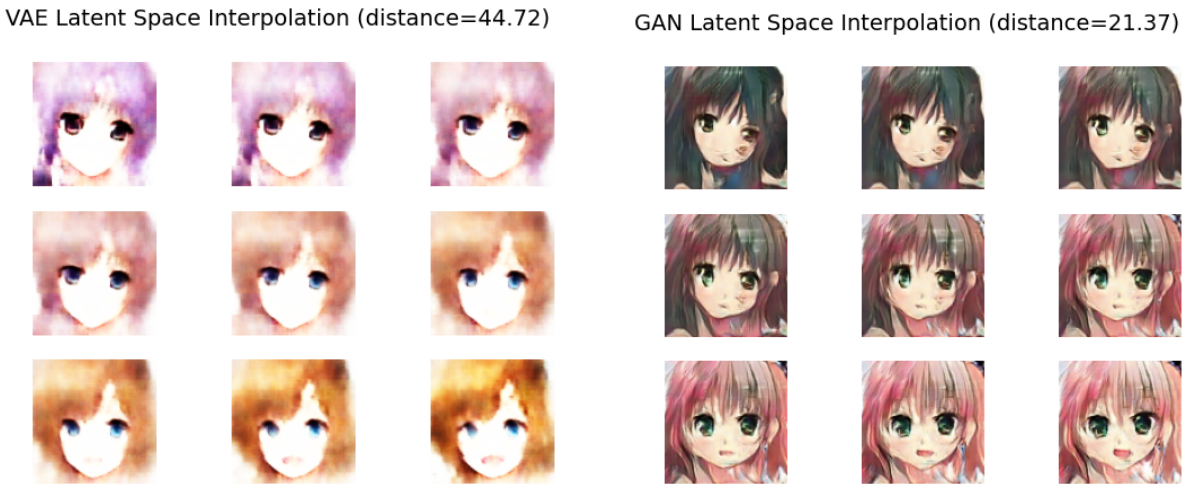
For the GAN, the number of tuneable hyperparameters also made a grid search infeasible. Instead, we also used trial and error. The set of hyperparameters is in Table 2.

Table 2: Hyperparameters for the GAN model

Parameter	Epochs	Batch size	Latent dim	Filters	Kernel	Upsample	LR	β_1
Value	100	64	256	256	3	5	2×10^{-4}	0.5

Interpolation

After training the models, two random input vectors with the size of the latent dimension were generated. The distance between the vectors was calculated and divided into 7 equal parts, creating 7 new vectors. In total, we now have 9 vectors evenly spaced apart. These vectors were given as input to the decoder to create 9 images (see Figure 1) that contain the interpolation between the two random input vectors. The first and last images are the random vectors; the 7 images in between are the interpolated vectors.



(a) Interpolation with VAE.

(b) Interpolation with GAN.

Figure 1: Comparison of latent space interpolation between VAE and GAN.

What we can derive from the images is that each interpolation gives a new face-like image, which indicates that each model has learned a smooth latent manifold where intermediate points correspond to coherent blends of the two endpoints, rather than random noise. This suggests that the generator captures consistent structures in the data and that latent directions encode meaningful visual changes.

GAN vs VAE: GAN interpolations show sharper edges and more coherent facial structure, while the VAE outputs are blurrier. The GAN shows smoother color blending.

1.5 Discussion of Working Principles

The VAE and GAN models operate on different principles to generate data.

The VAE is a probabilistic model that learns a smooth, continuous latent space. It consists of an encoder that maps input images to a distribution (defined by mean and variance) in the latent space, and a decoder that samples from this distribution to reconstruct the image. The loss function combines a reconstruction loss (ensuring that the output resembles the input) and a KL-divergence term (ensuring that the latent distribution approximates a standard Gaussian). This structure forces the latent space to be continuous, allowing for smooth interpolation between data points, though often resulting in slightly blurrier images.

The GAN, on the other hand, consists of two competing networks: a Generator that creates images from random noise, and a Discriminator that tries to distinguish between real and generated images. They are trained in a min-max game where the Generator tries to fool the Discriminator, and the Discriminator tries to correctly classify images. This adversarial process tends to produce sharper, more realistic images than VAEs, but the training can be unstable (e.g., mode collapse), and the latent space is not guaranteed to be smooth or continuous. Notably, GAN is not a covering model [1].

When the KL weight is too large in VAE, the posterior collapses and the generated images become overly similar—blurry but still globally coherent. Likewise, when the scaling coefficient applied to the latent vector exceeds 1, the generated samples lose diversity. If the KL term is dominated by the reconstruction loss, reductions in the total loss tend to yield sharp but semantically meaningless generated images.

In the VAE, the loss compares the input and output pixel-wise, meaning both tensors share the same shape $H \times W \times C$. This pixel-wise comparison is also channel-wise: each output channel (C_1, C_2, C_3) is matched to the corresponding input color channel (R, G, B). Consequently, the network learns to reconstruct $C_1 \to R$, $C_2 \to G$, and $C_3 \to B$ consistently across the image.

In our experiments, the VAE produced less diverse and blurrier outputs due to the explicit regularization of the latent space, while the GAN produced sharper images but struggled with random artifacts and disfiguration in earlier epochs.

2 Task 2: Sequence Modeling with RNNs

In this task, we studied encoder-decoder recurrent models for sequence-to-sequence learning. The objective was to develop neural networks capable of learning the principles behind simple arithmetic operations (addition and subtraction of two-digit integers). The problem involves mapping input sequences (e.g., '81+24' or '41-89') to output sequences (e.g., '105' or '-48').

This arithmetic problem is more complex than simple classification because the network cannot simply memorize solutions; it must learn the underlying mathematical rules to generalize to unseen queries.

We conducted the following experiments:

1. **Text-to-Text (T2T) Generalization**: We trained the model on decreasing proportions of the training data (75%, 50%, 25%, 10%, 5%) to evaluate its generalization capabilities and analyze the types of errors produced.

2. **Multimodal Mappings**: We increased the complexity by changing the input or output modalities:

- **Image-to-Text (I2T)**: The input consists of a sequence of 5 MNIST images, and the model generates the answer as text.
- **Text-to-Image (T2I)**: The input is a text sequence, and the model generates the answer as a sequence of 3 images.

3. **Network Depth Analysis**: We investigated the impact of adding additional LSTM layers to the encoder and decoder to improve performance.

For the T2T and I2T models, we measured generalization performance using test accuracy and error analysis. For the T2I model, we relied on visual inspection of the generated outputs.

2.1 Dataset Generation

The tools for generating the dataset were provided with the assignment. These tools included functions to generate images of operator symbols, combine them with digit images to construct query and result (label) images, and generate corresponding text-based versions. The dataset contains all possible arithmetic queries for integers up to 99, resulting in approximately 20,000 query-answer pairs.

2.2 Experiment 1: Text-to-Text Model

The architecture of the T2T model was provided with the assignment. The input queries (e.g., '12+34') are encoded as sequences of length 5, and the targets (e.g., '46 ') as sequences of length 3. Both are represented using one-hot encoding with a vocabulary of 13 characters (digits 0–9, '+', '-', and space). Thus, the input tensor has dimensionality $(N, 5, 13)$ and the output tensor $(N, 3, 13)$, where N is the batch size. The encoder consists of a single LSTM layer producing a final output vector of size 256. This vector is repeated 3 times using a `RepeatVector` layer layer to serve as the input sequence for the decoder. The decoder LSTM also produces 3 outputs of size 256. Finally, a dense layer with `softmax` activation is applied to each output timestep to produce character probabilities. The loss function is categorical cross-entropy. For the optimizer, Adam was chosen. Early stopping based on validation loss with restoring the best weights was used. A function to train the model on different training–test ratios was implemented to structure the experiment.

2.2.1 Hyperparameters

Since tuning the hyperparameters was not the goal of the assignment, we mainly focused on the effects of different train/test splits. We chose these commonly used values for our hyperparameters:

Hyperparameter	**Value**
Epochs	100
Batch size	128
Early stopping patience	10
Min delta	0 (Default)

Table 3: Hyperparameter settings for the T2T model.

2.2.2 Results

The results based on accuracy on the validation set and the test set are presented in Figure 2.

We observe that the model trained with the largest proportion of training data outperforms the other models. A steep drop in accuracy occurs when the proportion of training data drops below 50%.



Figure 2: Validation accuracy per split and test accuracy on different training-test splits in the data.

This suggests that there is a critical threshold for generalization between 50% and 25% training data (approximately 10,000 and 5,000 training examples). When the training set becomes too small, the model tends to memorize the training examples rather than learning the underlying arithmetic principles.

2.2.3 Error Analysis

For the analysis of the errors, we wrote a function to identify 6 types of errors. Each type description contains an example from a real prediction error:[1]

1. **Sign Error**: the prediction has a different sign compared to the label. (*Query '13-9' → True: 4, Predicted: -5 (diff: -9)*)
2. **Overflow Error**: the length of the output (number of digits) does not match the label. (*Query '22-9' → True: 13, Predicted: 1 (diff: -12)*)
3. **Carry/Borrow Error**: errors in how tens are carried over or borrowed when a column-wise operation cannot be represented with a single digit. (*Query '87-38' → True: 49, Predicted: 59 (diff: +10)*)
4. **Single Digit Error**: errors between 1–9. (*Query '38+91' → True: 129, Predicted: 136 (diff: +7)*)
5. **Magnitude Error**: the prediction is off by 10 or more but the error cannot be linked to a specific arithmetic principle. (*Query '90+9' → True: 99, Predicted: 10 (diff: -89)*)
6. **Format/Parse Error**: the output is non-numeric. (*Query '3-11' → True: -8, Predicted: -*)

Figure 3 visualizes the errors per type and per proportion of training data:

The graph clearly shows that single-digit errors are the most common errors, especially for the best-performing models (training proportions 75% and 50%). This suggests that for these models, individual recognition of digits is the most important factor when the prediction is wrong. Carry/borrow errors are the second most common in the best-performing models. This indicates difficulty with the carrying mechanism when adding or subtracting.

2.3 Experiment 2: Image-to-Text Model

For the image-to-text model (I2T), the T2T model was used as a basis. The following changes were applied: The input format was changed to accept a sequence of images of shape $(5, 28, 28, 1)$. The LSTM was replaced by 3 Convolutional LSTM layers (including batch normalization), which combine

[1] Taken from the output of the error analysis of the model trained on 25% training data.

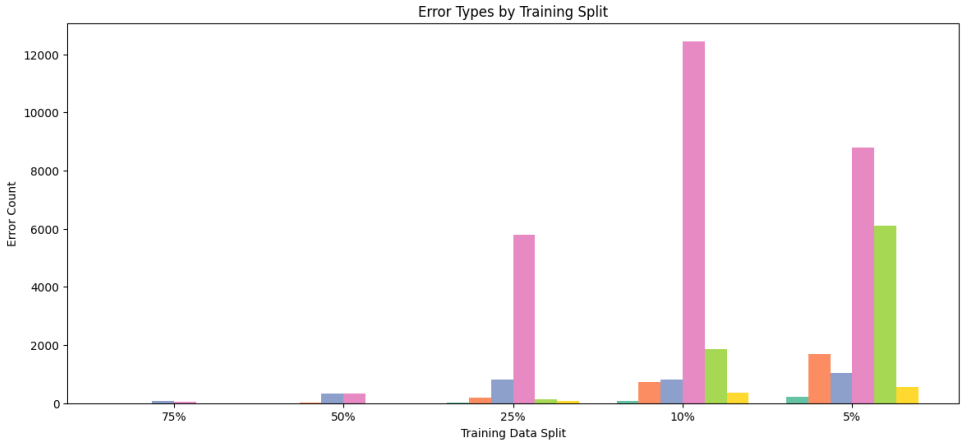


Figure 3: Error types per proportion of training data.

convolution to learn spatial features and recurrence to learn the temporal sequence [2]. Before passing the encoder output to the decoder, we added a layer to flatten the output into a 1D feature vector.

2.3.1 Hyperparameters

Since tuning the hyperparameters was not the goal of the assignment, we mainly focused on the effects of different train/test splits. We chose these commonly used values for our hyperparameters:

Hyperparameter	**Value**	**Explanation**
Epochs	100	Identical to the T2T model
Batch size	64	Reduced compared to T2T (128) due to increased memory requirements
Dropout rate	0.3 / 0.5	0.3 for ConvLSTM, 0.5 for Dense; added due to overfitting
Early stopping patience	10	Identical to the T2T model
Min delta	0 (Default)	Identical to the T2T model

Table 4: Hyperparameter settings for the I2T model.

2.3.2 Results

The results based on accuracy on the validation set and the test set are presented in Figure 4.

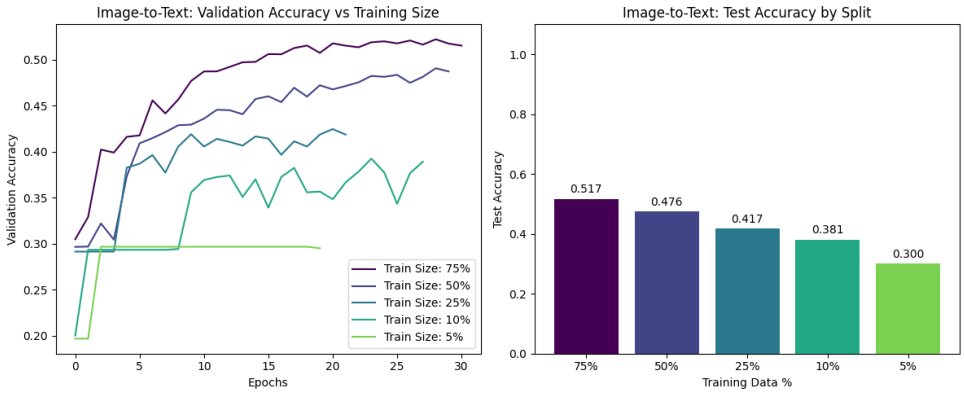


Figure 4: Validation accuracy per split and test accuracy on different training-test splits in the data.

We observe lower overall performance compared to the T2T model. The I2T model achieves approximately 50.79% accuracy with the 75% training split, significantly lower than the 99.29% achieved by

the T2T model at the same split. Combined with the fact that the task of the I2T model is much more complex than the task of the T2T model (requiring both visual recognition and arithmetic reasoning), we believe that the poor performance could be due to the added complexity and the limited amount of training data. The error types found (analyzed with a 75% proportion of training data) show that the I2T model makes all types of mistakes, suggesting that it did not fully grasp the arithmetic principles. Both the comparison of the accuracy and the error types are presented in Figure 5 and Figure 6.



Figure 5: Comparison of the test accuracy and error types per proportion of training data.

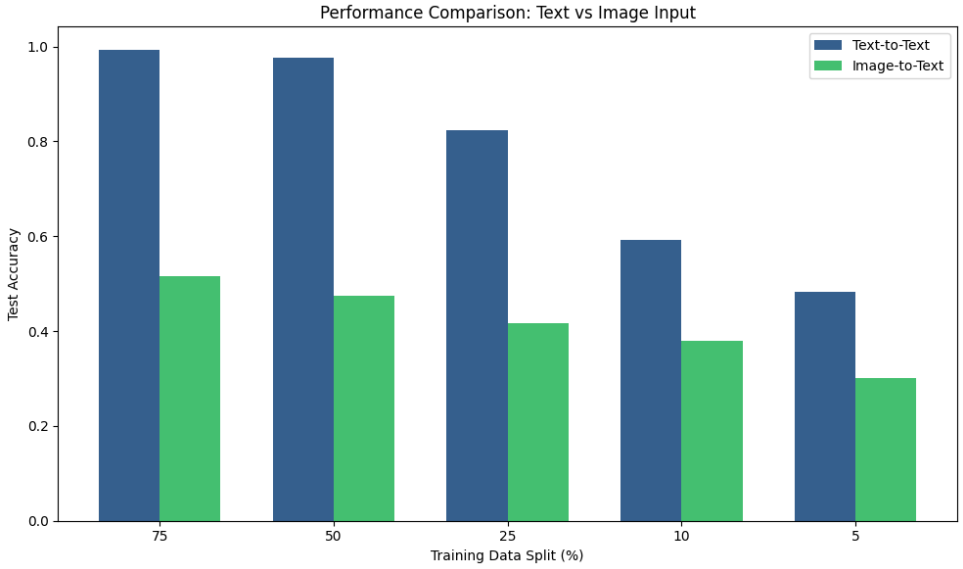


Figure 6: Performance comparison between Text-to-Text and Image-to-Text models across different training splits.

2.3.3 An Alternative Model

We also investigated an alternative architecture using a TimeDistributed CNN encoder with an LSTM decoder. Since CNNs are effective at recognizing handwritten digits and the TimeDistributed wrapper allows applying the same layer to each temporal slice of the input, this model is well-suited for capturing information from the sequence of 5 input images.

2.3.4 Results

This model achieved a test accuracy of 93.46% using 50% of the dataset to train the model. This represents a significant improvement over the ConvLSTM2D model, which achieved approximately 50.79% accuracy

even with 75% of the training data. This suggests that CNN-based architectures may be more suitable for this task, though detailed investigation is beyond the scope of this assignment.

2.4 Experiment 3: Text-to-Image Model

For the text-to-image model (T2I), the T2T model was used as a basis. The output layer was adjusted so that, instead of producing character probabilities, it reconstructs image frames through a sequence of learned spatial feature maps and transposed convolutions. Starting with a dense projection to a compact $7 \times 7 \times 64$ feature map, the decoder then upsamples and refines this representation to generate the final 28×28 image frames. For the loss function, MSE was used because it is more stable with the continuous grayscale pixel values the model reconstructs.

2.4.1 Hyperparameters

Since tuning the hyperparameters was not the goal of the assignment, we mainly focused on the effects of different train/test splits. We chose these commonly used values for our hyperparameters:

Hyperparameter	**Value**	**Explanation**
Epochs	100	Identical to the T2T model
Batch size	64	Reduced compared to T2T (128)
Gradient clipping	1.0	Introduced to prevent exploding gradients
Early stopping patience	10	Identical to the T2T model
Min delta	0 (Default)	Identical to the T2T model

Table 5: Hyperparameter settings for the T2I model.

2.4.2 Results

Quantitative evaluation of the T2I model is difficult without an additional model to convert the output images back into characters. We attempted to train a digit classifier to evaluate the generated images, but this led to ambiguities: A low score could be due to either poor image generation (unidentifiable digits) or incorrect calculations (identifiable but incorrect digits). Furthermore, without ground truth for the *intended* generated image, it is difficult to distinguish between these modes of error.

Consequently, we opted for a visual analysis of the results (see Figure 7 in Appendix D). The 8 examples we observed resulted in blurry images with softer edges than the true images. This might be due to the MSE loss that produces average pixel values. While the positioning of the numbers is correct, the model seems to have difficulty distinguishing between, e.g., 3, 7, and 8. A more robust digit classifier model would enable more extensive quantitative analysis of the entire output and would be our suggestion for future work. We included the preliminary results of the digit classifier in Figure 8 (Appendix E).

2.5 LSTM Layer Analysis

In this section, we explored the effect of adding extra LSTM layers to the T2T model and the I2T model (both trained on 75% of the dataset).

We tested all combinations of 1–2 LSTM layers in both the encoder and decoder parts of the T2T model. To implement the stacked LSTM configurations (e.g., 2 layers), we set `return_sequences=True` on the first LSTM layer to pass the full sequence to the second layer. We used the number of carry errors as an indicator of how well the model grasps the underlying arithmetic principles, alongside test accuracy for overall performance.

For the I2T model, the baseline encoder consists of a stack of 3 ConvLSTM layers. In the deeper configuration, we increased the depth of the encoder to 4 ConvLSTM layers. The decoder configurations remain the same as in the T2T model (1 vs. 2 LSTM layers).

2.5.1 Results

The results for the T2T model (based on 100 epochs) are shown in the table below.

Configuration	**Test Accuracy**	**Carry Errors**	**Reduction of Carry Errors**
1 + 1 (baseline)	99.29%	61	–
2 + 1	99.77%	10	83.6%
1 + 2	99.43%	31	49.2%
2 + 2	99.84%	0	100%

Table 6: Results of different LSTM configurations for the T2T model.

As shown in Table 6, adding more LSTM layers decreases the carry errors and increases the model accuracy. Since the accuracy of the baseline model is already above 99%, the increase in accuracy is minimal.

The results for the I2T model (based on 100 epochs) are shown in the table below.

Configuration	**Test Accuracy**	**Carry Errors**	**Reduction of Carry Errors**
3 + 1 (baseline)	50.79%	198	–
4 + 1	54.53%	137	30.8%
3 + 2	52.33%	162	18.2%
4 + 2	33.33%	221	−11.6%

Table 7: Results of different LSTM configurations for the I2T model.

Contrary to the T2T model, the I2T configurations do not show a consistent improvement when additional LSTM layers are added. The baseline model reaches a test accuracy of 50.79% with 198 carry errors. Adding one encoder layer (4 + 1) provides the strongest improvement: the test accuracy increases to 54.53% and the number of carry errors decreases by 30.8%.

The 3 + 2 configuration gives only a small increase in test accuracy (52.33%) and a modest reduction of carry errors (18.2%). Increasing the depth further (4 + 2) results in a deterioration of performance: the test accuracy drops to 33.33% and carry errors rise to 221, corresponding to a negative reduction score of −11.6%.

Overall, the I2T results show that adding layers does not reliably enhance the model. Only the 4 + 1 configuration produces a measurable improvement, while the 4 + 2 configuration reduces accuracy and worsens the number of carry-related mistakes.

2.5.2 Summary of LSTM Analysis

In summary, the performance of RNNs in this setting relies on balancing input complexity, model flexibility, and dataset size. A highly flexible model (like the stacked I2T) trained on limited data tends to overfit rather than learn the underlying arithmetic rules, leading to poor generalization. We observed that while adding depth helped the simpler T2T task, it was detrimental to the more complex I2T task given the available data.

3 Discussion and Conclusion

3.1 Generative Models: Key Insights

Our experiments with VAEs and GANs highlighted the fundamental trade-offs between these two families of generative models. The VAE training was stable and produced a smooth, continuous latent space, which was evident in the continuous interpolation results. However, the generated images suffered from blurriness due to the probabilistic nature of the reconstruction loss. In contrast, the GAN produced significantly sharper and more realistic images, but proved more difficult to train and exhibited instability. We found the latent dimension to be a critical hyperparameter. A dimension that is too small restricts the model's ability to capture the diversity of the dataset, while a dimension that is too large results in a sparse latent space, making meaningful interpolation and sampling more difficult.

3.2 Sequence Modeling: Key Insights

The core challenge of the arithmetic task lies in learning the underlying function (addition/subtraction) rather than memorizing a lookup table. Our T2T experiments showed that there is a *critical mass* of training data (around 50% of the dataset) required for the model to transition from memorization to generalization. Below this threshold, the model failed to learn complex rules like carrying and borrowing, resulting in a high frequency of specific error types. The comparison between Text-to-Text (T2T) and Image-to-Text (I2T) models demonstrated the significant cost of processing raw visual data. The T2T model, dealing with symbolic inputs, achieved near-perfect accuracy (>99%) with sufficient data. The I2T model, which had to simultaneously learn to recognize digits and perform arithmetic, plateaued at a much lower accuracy ($\sim$54.53% at best). This suggests that for complex reasoning tasks, decoupling perception (digit recognition) from reasoning (arithmetic) might be a more effective strategy than end-to-end training on limited data.

3.3 Conclusion

In this assignment, we successfully investigated the capabilities of deep generative models and recurrent neural networks. We showed that both VAEs and GANs can generate new data, but serve different purposes depending on whether the priority is a smooth latent space or image quality. In sequence modeling tasks, we showed that RNNs are capable of learning abstract mathematical rules, but this ability is severely limited by the input medium and the volume of training data. The ability to generalize to unseen queries requires a model architecture and dataset size that discourage memorization.

3.4 Weaknesses and Future Work

Our I2T models relied on a basic ConvLSTM architecture, which struggled to extract features effectively from the image sequences. Additionally, the evaluation of the Text-to-Image (T2I) model was purely qualitative, limiting our ability to quantitatively evaluate its performance. To improve the T2I model assessment, training a separate digit classifier to automatically evaluate the generated equations would provide a quantitative metric for success. Finally, replacing the LSTM cells with Transformer architectures (using self-attention) would likely improve performance on longer sequences and help the model better capture the dependencies required for multi-digit arithmetic.

References

[1] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

[2] Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. Convolutional lstm network: A machine learning approach for precipitation nowcasting. *Advances in neural information processing systems*, 28, 2015.

A Theory

A.1 The Theory Behind VAE

The VAE treats image generation as a probabilistic latent-variable problem. It assumes a simple Gaussian prior $p(z) = \mathcal{N}(0, I)$ and models images through the joint distribution $p_\theta(x, z) = p_\theta(x|z)p(z)$. The true posterior $p_\theta(z|x)$ is intractable, so the encoder learns an approximation $q_\phi(z|x)$. Training maximizes the Evidence Lower Bound (ELBO),

$$\text{ELBO} = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p(z)),$$

which balances reconstruction fidelity and latent regularity. In the implementation used here, the reconstruction term is realized as a pixel-wise Binary Cross-Entropy (BCE) loss between the input image (scaled to $[0, 1]$) and the decoder output (sigmoid activation), while the KL-Divergence

$$D_{KL} = -\frac{1}{2}\sum_k (1 + \log \sigma_k^2 - \mu_k^2 - \sigma_k^2)$$

is added through a custom layer and normalized by the number of pixels ($96 \times 96 \times 3$). This normalization places the KL term on the same per-pixel scale as the reconstruction loss. The resulting VAE objective therefore becomes

$$L_{\text{VAE}} = L_{\text{BCE}}(x, \hat{x}) + D_{KL}/\text{data shape},$$

corresponding to a standard β-VAE with $\beta = 1$ but with both terms numerically balanced.

Here, x denotes the pack of all the real images in a batch and $\hat{x}$ (sometimes written as x' or x^*) denotes the fake ones. We also tried weighting the KL-loss term by the tunable hyperparameter β, manually tuned differently in some epochs. Once trained, sampling z from the posterior q and decoding it produces new images consistent with the learned distribution.

Reconstruction loss trains both encoder and decoder. The decoder learns to reconstruct images from latent vectors. The encoder learns to produce latent vectors whose local posterior distributions preserve useful visual information (color, shape, structure). This term encourages local fidelity: output $\approx$ input.

KL regularization trains only the encoder. It pushes each local posterior $q_\phi(z|x)$ toward the global prior $\mathcal{N}(0, I)$, yielding a globally smooth, continuous, and well-organized latent space. This term prevents the encoder from "cheating" by memorizing images and ensures correct sampling, because all posteriors collectively behave like a global Gaussian structure. The decoder is not affected by the KL term; it never sees μ or σ. Note that if this term is down-weighted by choosing a very small β, then the reduction of loss does not necessarily mean that the model is generating globally meaningful images.

A.2 The Theory Behind GAN

The GAN does not define an explicit likelihood—it is not a probabilistic generator. Instead, it trains a generator $G_\theta(z)$ and a discriminator $D_\phi(x)$ in an adversarial game. The GAN does not use an encoder unit, since the generator maps latent noise (sampled from $\mathcal{N}(0, I)$) to images using a deconvolutional network with a tanh output layer, so the training data are first rescaled to the range $[-1, 1]$. The discriminator is a convolutional binary classifier with a sigmoid output, estimating the probability of an image being real, $D_\phi(x) \in [0, 1]$. It is trained with BCE loss to output 1 for real images and 0 for generated (fake/synthesized) ones:

$$L_D = E_{x \sim p_{\text{data}}}[-\log D_\phi(x)] + E_{z \sim p(z)}[-\log(1 - D_\phi(G_\theta(z)))].$$

For generator training, the discriminator's weights are frozen and all target labels are set to 1, producing the non-saturating loss

$$L_G = -E_{z\sim p(z)}[\log D_\phi(G_\theta(z))],$$

which encourages the generator to make its samples indistinguishable from real images. Both networks are optimized with Adam (tunable by learning rate and β_1 momentum), a configuration known to improve GAN stability.

Our GAN uses the non-saturating generator loss. This form is preferred over the original "saturating" loss, $L_G^{\text{sat}} = E_{z\sim p(z)}[\log(1 - D(G(z)))]$, because the classic loss saturates when $D(G(z)) \approx 0$, producing near-zero gradients and causing the generator to stop learning. In contrast, the non-saturating loss yields strong gradients even when the discriminator is confident, leading to faster and more stable GAN training.

B CAE Grid Search Strategy

Changing the random seed altered the loss only in the fourth decimal place, whereas changing the hyperparameters affected the third decimal after 10 epochs. This suggests that running 10 epochs is sufficient for grid-search comparisons, assuming that early training consistently drives all tested settings toward the same local minimum.

As the input image satisfies $width = height = H$, and the number of downsampling and upsampling layers is U, the bottleneck dimensionality is $D_{\text{bn}} = \left(H//2^U\right)^2 \times C$, where C denotes the number of filters (channels) at the deepest layer, and 2 is the stride. This means for a latent dimension of L, only the fraction L/D_{bn} of the representational capacity can pass through the bottleneck. Thus, for $H = 64 \sim 96$, we keep $U \leq 5$ to avoid an overly restrictive bottleneck. Also, we keep $L \leq 512$ to prevent too large a fraction of information from bypassing the neck, which would undermine the regularizing effect of the bottleneck. Using 96×96 images instead of 64×64 allows us to set $U = 5$ for improved reconstruction quality (compared to $U = 4$), while keeping $H//2^U$ sufficiently large.

Note that although smaller batch size $B = 16$ reduces the loss more effectively, it requires many updates per epoch (N_i/B) for N_i training images. Since two random batches drawn from a large dataset differ substantially if they are too small, the model may wander at each update, increasing the risk of getting trapped in local minima or plateaus. For this reason, we prefer $B = 32$ over 16.

Increasing either the latent dimension or the number of filters raises the model capacity and therefore the training time, with runtime being more sensitive to the number of filters than to the latent size. In this capacity–time trade-off, the optimal setting would be the final configuration. Larger batch sizes (B) not only require more memory but also slow training substantially, since the loss is computed over the entire batch at once.

A kernel size of $k = 3$ with stride 2 is optimal: it preserves a one-row and one-column overlap between neighboring patches while remaining small enough to capture fine-grained patterns.

C Hyperparameter Interpretations for VAE and GAN

VAE

- **Size of the latent space**: If it is very low, causes underfitting, blurry reconstructions, and loss of fine details. If it is very high, causes overfitting, weak KL regularization, and unstable latent structure. We suggested **64** $\in \{32, \ldots, 128\}$.
- **Number of filters**: controls representational capacity in (de-)convolutional channels. If it is very low, causes weak feature extraction and low-detail, washed-out reconstructions. If it is very high, causes unnecessary memory usage and slower training without clear gains, and capturing noises as patterns. We suggested **128** for 64–96px images.
- **Number of downsampling and upsampling layers** [2]: depth of encoder and decoder networks. If they are very low, they cause shallow representations and weak abstraction of structure. If they are very high, they cause excessive compression and can degrade reconstruction quality. We suggested **5** layers for each, making a $3 \times 3 \times 128$ bottleneck for $96 \times 96 \times 3$ real images with 128 filters.
- **Learning rate**: step size for Adam optimizer controlling update speed. If it is very low, causes slow convergence, stuck training, and overly smooth images. If it is very high, causes training instability, KL collapse, or exploding loss. We suggested 10^{-4} **to** 10^{-3}.
- **Batch size**: number of samples processed per weight-update step. If it is very low, causes noisy gradients and unstable KL behaviour. If it is very high, causes smoother gradients but requires more memory and may reduce stochasticity. We suggested **64** $\in \{32, \ldots, 128\}$.
- **Number of epochs**: number of full passes through the dataset during training. If it is very low, causes incomplete convergence and blurry or noisy reconstructions. If it is very high, causes diminishing returns and possible over-regularization. We suggest **150–350** epochs, and tried about 200 epochs.
- **Coefficient**: a scaling factor applied to sampled z vectors during generation. If it is very low, causes low sample diversity and repetitive images close to the mean. If it is very high, causes unrealistic, noisy, or heavily distorted generated images. We suggested **2–4**.
- $\boldsymbol{\beta}$: weights KL-loss over BCE reconstruction loss in the total loss. If β is increased, reconstructed images become blurrier, since the more-penalized decoder is forced toward a unimodal $N(0, I)$, leaving less capacity for fine details (lower fidelity, lower texture richness, washed-out colors). But generated images become more realistic (globally coherent, fewer artifacts). We suggested **0.5–1.5**.

GAN

- **Latent dimension**: size of the generator input noise vector z. If it is very low, the generator has too few degrees of freedom, causing limited diversity and mode collapse (*similar faces repeated with small variations*). If it is very high, the generator may become harder to train and can waste capacity on noise-like variations. For 96×96×3 images, we suggested **64–256**.
- **Number of filters**: We suggested **128** filters for both generator and discriminator, dealing with 96px images.

[2] In the original notebook on BrightSpace that uses 64x64 images, the VAE's bottleneck was like "$2 \times 2 \times 128 \rightarrow latent_dim \rightarrow 4 \times 4 \times 64$". We fixed this mismatch in our code by altering the reshape() function, so that the bottleneck in ours is like "$3 \times 3 \times 128 \rightarrow latent_dim \rightarrow 3 \times 3 \times 128$" for our 96x96 images

- **Number of downsampling and upsampling layers**: Same design choices as in the VAE. Note that GANs do not form a shared bottleneck: the discriminator downsamples toward a real/fake decision, while the generator independently upsamples from a drawn noise latent vector.
- **Learning rate** for Adam optimizer: We suggested $2 \times 10^{-4} \in [5 \times 10^{-5}, 5 \times 10^{-4}]$.
- $\boldsymbol{\beta_1}$ (Adam's momentum): exponential decay rate for the first moment in Adam. If it is very high (close to 0.9), often destabilizes GAN training (overshooting and oscillations). If it is very low, updates become more "SGD-like" with less smoothing but may be more stable for adversarial training. We suggested $\mathbf{0.5} \in [0.4, 0.7]$.
- **batch_size**: We suggested **64** as a default, with a reasonable range **32**–**128**.
- **Number of epochs**: For this setup, we suggested starting from **30** epochs and visually monitoring generated samples after each epoch to decide whether to continue. Totally we used about 250 epochs.

  Playing with Adam's hyperparameters, like slightly increasing momentum β_1 while simultaneously decreasing the learning rate every 10–20 epochs, helped the model escape local minima and plateaus. The intuition is that lower learning rates provide stability, whereas temporarily higher momentum injects additional inertia that may push the optimization trajectory out of flat regions.

D Generated versus True Images

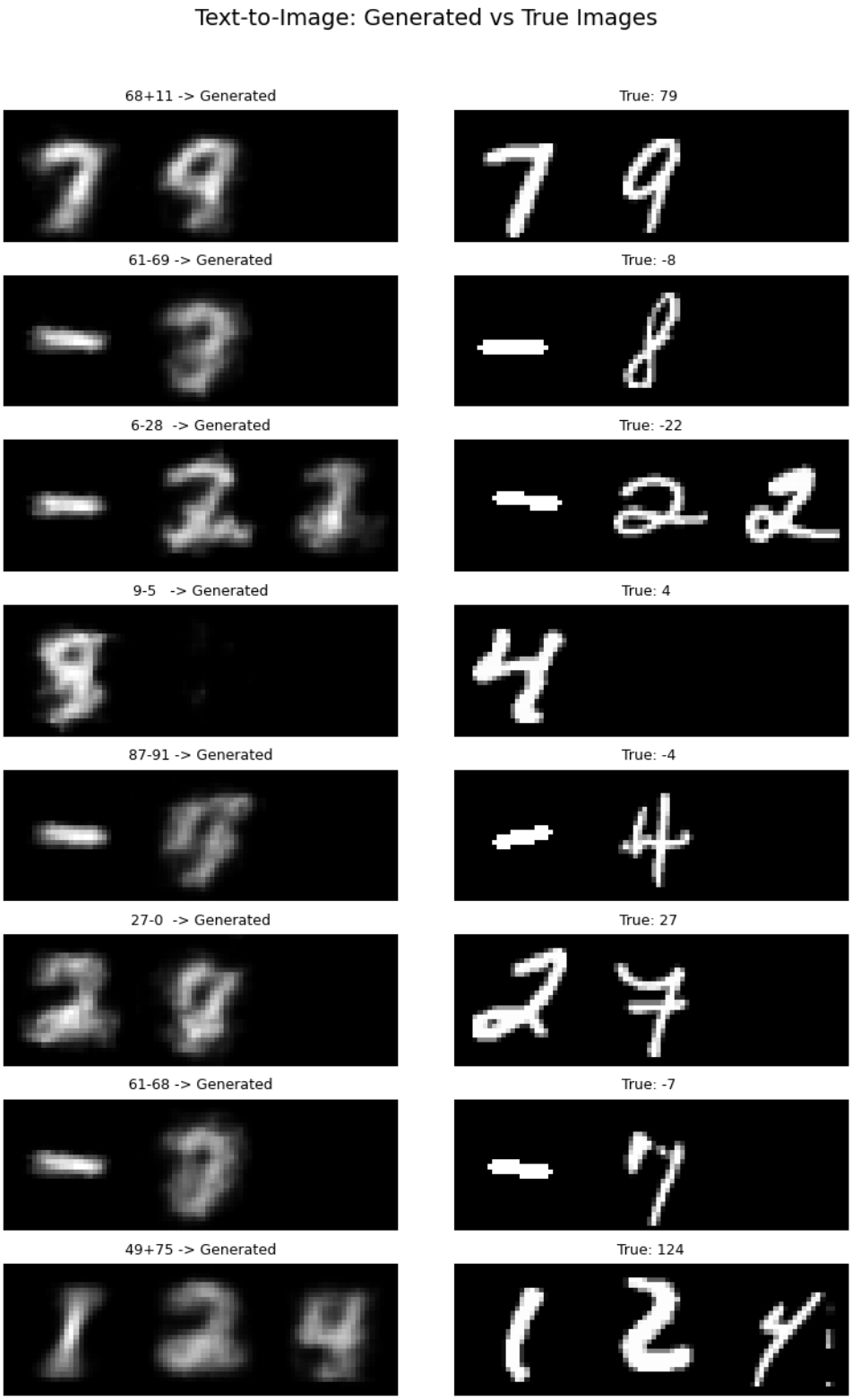


Figure 7: Generated versus True images from the Text-to-Image model.

E Digit Classifier Evaluation

To quantitatively evaluate the Text-to-Image model, we trained a separate digit classifier on the MNIST-style images generated by our dataset creation function. The classifier achieved high accuracy on real images, but performed significantly worse on the generated images, indicating potential quality issues in the generation process.

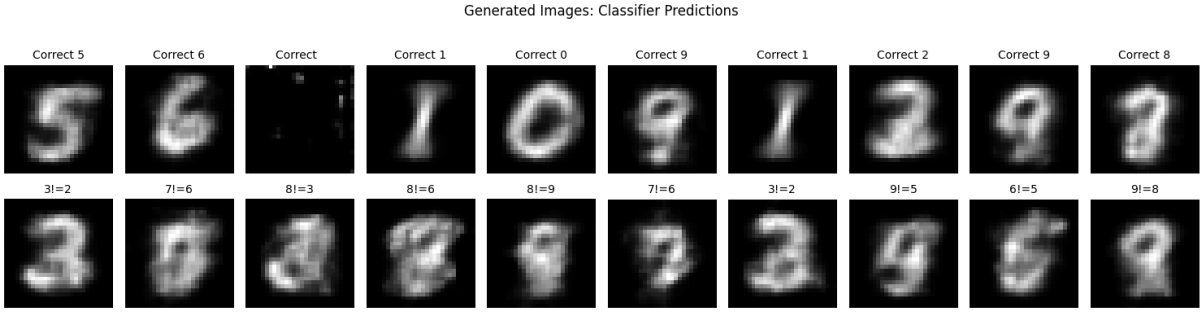


Figure 8: Evaluation of generated images using a digit classifier. Top row: Correctly classified generated images. Bottom row: Incorrectly classified generated images.